

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ  
ОДЕСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ І.І.МЕЧНИКОВА  
Факультет математики, фізики та інформаційних технологій  
Кафедра математичного забезпечення комп'ютерних систем

КУРСОВИЙ ПРОЄКТ  
з освітнього компоненту «Програмування»  
на тему: СТВОРЕННЯ ІЄРАРХІЇ КЛАСІВ НА ТЕМУ «ЗАПЧАСТИНИ»

Студента 1 курсу, денна форма навчання  
спеціальність F7 «Ком'ютерна інженерія»

Геращенко Наталії Тигранівни

Керівник: к.ф.- м.н., доц. Антоненко О. С.

Національна шкала: \_\_\_\_\_

Кількість балів: \_\_\_\_\_

ECTS: \_\_\_\_\_

Дата захисту: \_\_\_\_\_

Члени комісії \_\_\_\_\_ Антоненко О.С.  
(підпис) (прізвище та ініціали)

\_\_\_\_\_ Шестопапов С.В.  
(підпис) (прізвище та ініціали)

\_\_\_\_\_ \_\_\_\_\_  
(підпис) (прізвище та ініціали)

## ЗМІСТ

|                            |    |
|----------------------------|----|
| ВСТУП .....                | 3  |
| ЗАВДАННЯ .....             | 4  |
| 1. ТЕОРЕТИЧНА ЧАСТИНА..... | 5  |
| 1.1 Поняття ООП.....       | 5  |
| 2. ПРАКТИЧНА ЧАСТИНА ..... | 6  |
| 2.1 Опис класів .....      | 6  |
| 2.2 Інтерфейс.....         | 8  |
| ВИСНОВКИ.....              | 15 |
| СПИСОК ЛІТЕРАТУРИ.....     | 16 |

## ВСТУП

Мета роботи – створити ієрархію класів на тему «Запчастини» та застосувати її для розв'язання прикладної задачі, визначеної у варіанті завдання. Під час виконання роботи необхідно використовувати основні принципи об'єктно-орієнтованого програмування: інкапсуляцію, успадкування та поліморфізм. Згідно із завданням потрібно реалізувати систему взаємопов'язаних класів, що описують структуру та роботу заводу з виробництва деталей, а також складського комплексу для їх зберігання.

У програмі потрібно реалізувати ієрархію класів на основі відношень успадкування (створення класу готового продукту на базі класу запчастини). Для реалізації динамічного поліморфізму слід використовувати віртуальні методи (зокрема, для виведення інформації про об'єкти) та віртуальні деструктори для коректного очищення динамічної пам'яті.

Також у роботі необхідно передбачити обробку виняткових ситуацій для контролю фізичних параметрів складу (ваги та об'єму деталей) і валідації артикулів під час взаємодії користувача з консольним інтерфейсом.

## ЗАВДАННЯ

### **Варіант 25. Створення ієрархії класів на тему «Запчастини»**

У роботі необхідно реалізувати класи, що описують структуру виробництва та складського обліку деталей. Передбачити можливість роботи з базовим класом запчастин та похідним класом продуктів, зберігання інформації про їх назву, артикульний номер, фізичні характеристики, а також економічні показники (вартість і час виробництва). Для управління об'єктами необхідно розробити класи «Завод» та «Склад». Також необхідно реалізувати взаємодію між класами (зокрема, виробництво деталі за артикулом та її відвантаження на зберігання), перевірку коректності даних (контроль вільного місця на складі) та виконання основних операцій, пов'язаних із функціонуванням розробленої системи через консольне меню.

## 1. ТЕОРЕТИЧНА ЧАСТИНА

### 1.1 Поняття ООП

Об'єктно-орієнтоване програмування (ООП) – це метод розробки програмного забезпечення, заснований на поданні програми у вигляді сукупності взаємодіючих об'єктів. Кожен об'єкт є екземпляром певного класу, а самі класи утворюють ієрархію наслідування. Програмісти спочатку проєктують класи, які описують стан та поведінку сутностей, а вже на їхній основі під час виконання програми створюються конкретні об'єкти.

Концептуальною базою такого підходу є об'єктна модель, яка спирається на три фундаментальні принципи:

- а) інкапсуляція;
- б) наслідування;
- с) поліморфізм.

Інкапсуляція – механізм, який об'єднує дані та методи, що працюють з цими даними, в єдину структуру (клас), а також захищає їх від прямого зовнішнього втручання або некоректного використання. У розробленій системі інкапсуляція застосовується для приховування внутрішніх характеристик запчастин, місткості складу та виробничих каталогів заводу. Доступ до цих полів здійснюється виключно через передбачені методи.

## 2. ПРАКТИЧНА ЧАСТИНА

### 2.1 Опис класів

Діаграму класів наведено в додатку А. Розглянемо більш детально структуру кожного класу.

1) Базовий клас «SparePart»:

a) Захищені поля : name, number, weightGrams, volumeCm3

b) Конструктори:

– SparePart (ініціалізує назву, ідентифікаційний номер, вагу та об'єм пакування)

c) Методи:

– getWeightKg – метод, що повертає вагу у кілограмах

– getVolumeM3 – метод, що повертає об'єм упаковки у кубічних метрах

– printInfo – метод для виведення базової інформації про запчастину

2) Клас «Product», нащадок класу «SparePart»:

a) Приватні поля: cost, productionTimeDays

b) Конструктори:

– Product (ініціалізує успадковані властивості базового класу та нові специфічні поля продукту – вартість і час)

c) Методи:

– printInfo – метод виведення розширеної інформації про продукт (доповнює базову інформацію даними про запчастину)

3) Клас «Factory»:

a) Приватні поля: catalog

b) Конструктори:

- Конструктор за замовчуванням

с) Методи:

- addProductToCatalog – додає новий продукт до виробничого каталогу заводу
- printCatalog – метод виведення інформації про всі продукти, які завод здатний виробляти
- produceSparePart – метод «Виробити запчастину» (моделює процес створення готової деталі)

4) Клас «Warehouse»:

а) Приватні поля: name, address, dailyRentCost, totalVolumeM3, currentVolumeM3, storage (вектор вказівників на базовий клас vector<SparePart\*>)

б) Конструктори:

- Warehouse (ініціалізує назву, адресу локації, вартість оренди за добу та загальний максимальний об'єм складу; поточний об'єм currentVolumeM3 за замовчуванням встановлюється у 0.0)

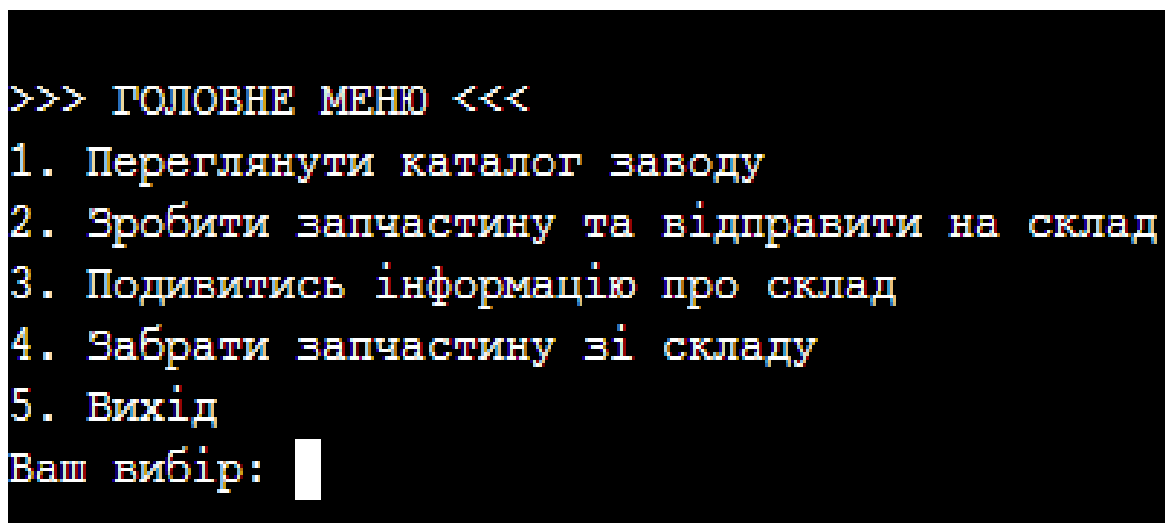
с) Методи:

- printInfo – метод виведення інформації про загальний стан складу
- addPart – метод додавання запчастини на склад (фізичне розміщення об'єкта у масив storage)
- removePart – метод вилучення (забирання) запчастини зі складу

## 2.2 Інтерфейс

Під час запуску програми користувач бачить вступне вікно (Рис 2.1). Це меню пропонує обрати, з якою частиною системи він бажає працювати: Заводом-виробником чи Логістичним складом.

Користувач потрапляє сюди коли запускає програму. Якщо користувач замість літери або неіснуючого пункту меню введе некоректний символ, програма виведе повідомлення про помилку за допомогою функції `clearInput()` і повернеться до очікування введення.



```
>>> ГОЛОВНЕ МЕНЮ <<<
1. Переглянути каталог заводу
2. Зробити запчастину та відправити на склад
3. Подивитись інформацію про склад
4. Забрати запчастину зі складу
5. Вихід
Ваш вибір: █
```

Рисунок 2.1 – Головне меню програми

Вибір 1: Переглянути каталог заводу:

Для кожної доступної деталі (об'єкта класу `Product`) на екран будуть виведені її успадковані базові характеристики (назва запчастини, номенклатурний номер, вага, об'єм упаковки), а також специфічні для заводу дані: вартість виробництва у доларах (`cost`) та час, необхідний на її виготовлення у днях (`productionTimeDays`) (Рис 2.2).



```

Ваш вибір: 1

--- КАТАЛОГ ЗАВОДУ ---
Двигун V8      | № ENG-001   | Вес: 150   кг | Об'єм: 2     м^3 | Вартість: $5000 | Виробництво: 14 дн.
Гальмівні колодки | № BRK-099   | Вес: 1.5   кг | Об'єм: 0.005 м^3 | Вартість: $45   | Виробництво: 1 дн.
Фара передня  | № LGH-102   | Вес: 2.5   кг | Об'єм: 0.015 м^3 | Вартість: $120   | Виробництво: 2 дн.
-----

>>> ГОЛОВНЕ МЕНЮ <<<
1. Переглянути каталог заводу
2. Зробити запчастину та відправити на склад
3. Подивитись інформацію про склад
4. Забрати запчастину зі складу
5. Вихід
Ваш вибір:

```

Рисунок 2.2 – Вікно після вибору 1 пункту

Після виведення каталогу заводу програма знову запропонує головне меню, очікуючи наступний вибір користувача. Користувач може переглянути те, що його цікавить, або завершити, натиснувши «5».

Вибір 2: Зробити запчастину та відправити на склад:

В цьому випадку користувач повинен ввести артикул (номер) запчастини з каталога, яку він бажає зробити на відправити на склад (Рис 2.3).

```

Ваш вибір: 2
Введіть артикул (номер) запчастини з каталога: 

```

Рисунок 2.3 – Вікно після вибору 2 пункту

Після введення коректного номера, програма повідомить користувача, що запчастина з відповідним номером успішно додана на склад та запропонує головне меню (Рис 2.4).

```

Ваш вибір: 2
Введіть артикул (номер) запчастини з каталога: BRK-099
Запчастина успішно додана на склад.

>>> ГОЛОВНЕ МЕНЮ <<<
1. Переглянути каталог заводу
2. Зробити запчастину та відправити на склад
3. Подивитись інформацію про склад
4. Забрати запчастину зі складу
5. Вихід
Ваш вибір:

```

Рисунок 2.4 – Вікно у випадку введення коректного номера запчастини

Якщо користувач введе номер, якого немає в каталозі, програма видасть повідомлення, що запчастина з данним артикулом не знайдена в каталозі заводу, та поверне користувача до меню (Рис 2.5).

```

Ваш вибір: 2
Введіть артикул (номер) запчастини з каталога: DFH-007
[ЗНАЙДЕНА ПОМИЛКА] Запчастина з артикулом DFH-007 не знайдена в каталозі заводу!

>>> ГОЛОВНЕ МЕНЮ <<<
1. Переглянути каталог заводу
2. Зробити запчастину та відправити на склад
3. Подивитись інформацію про склад
4. Забрати запчастину зі складу
5. Вихід
Ваш вибір: 

```

Рисунок 2.5 – Вікно у випадку введення некоректного номера запчастини

У випадку, якщо користувач додасть на склад запчастини, які будуть загалом займати більше об'єму ніж містить сам склад, програма повідомить користувача про помилку (недостатньо місця на складі) та скасує операцію (Рис 2.6). Після чого поверне головне меню для подальшої роботи.

```

Ваш вибір: 2
Введіть артикул (номер) запчастини з каталога: ENG-001
[ЗНАЙДЕНА ПОМИЛКА] Недостатньо місця на складі! Операцію скасовано.

>>> ГОЛОВНЕ МЕНЮ <<<
1. Переглянути каталог заводу
2. Зробити запчастину та відправити на склад
3. Подивитись інформацію про склад
4. Забрати запчастину зі складу
5. Вихід
Ваш вибір:

```

Рисунок 2.6 – Вікно у разі нестачі місця на складі

Вибір 3: Подивитись інформацію про склад:

Коли користувач перейде до 3 пункту, на екран буде виведено поточний стан логістичного вузла (об'єкта класу Warehouse), та початкове меню.

Користувач побачить структуровану інформацію, що базується на приватних полях класу (Рис 2.7). Програма наочно покаже загальний максимальний об'єм складу (totalVolumeM3) та об'єм, який вже зайнято деталями (currentVolumeM3).

Також буде виведена загальна кількість деталей, які зараз зберігаються у векторі storage.

```

Ваш вибір: 3

=== ІНФОРМАЦІЯ ПРО СКЛАД ===
Назва: Центральний Хаб | Адреса: Промислова 12
Оренда: $200/доба
Об'єм: 0 / 5 м^3
Запчастин у наявності: 0
=====

```

Рисунок 2.7 – Перегляд інформації про склад

Коли користувач додає запчастину на склад (Рис 2.4), інформація про неї (об'єм, наявність) буде додана в меню «Інформація про склад» (Рис 2.8). Також можна побачити вміст складу, в якому будуть додані користувачем запчастини.

```
Ваш вибір: 3

=== ІНФОРМАЦІЯ ПРО СКЛАД ===
Назва: Центральний Хаб | Адреса: Промислова 12
Оренда: $200/доба
Об'єм: 2 / 5 м^3
Запчастин у наявності: 1

Вміст складу:
Двигун V8      | № ENG-001      | Вес: 150      кг | Об'єм: 2      м^3 | Вартість: $5000      | Виробництво: 14 дн.
```

Рисунок 2.8 – інформація про склад та додані до нього запчастини

Так само, якщо користувач забажає забрати якусь запчастину зі складу, це відобразиться на загальній інформації про склад (зменшиться наявність та зайнятий об'єм).

Вибір 4: Забрати запчастину зі складу:

Користувач може «забрати» (відвантажити) товар зі складу за його артикулом (Рис 2.9). Програма здійснює пошук у векторі.

```
>>> ГОЛОВНЕ МЕНЮ <<<
1. Переглянути каталог заводу
2. Зробити запчастину та відправити на склад
3. Подивитись інформацію про склад
4. Забрати запчастину зі складу
5. Вихід
Ваш вибір: 4
Введіть артикул запчастини для видачі:
```

Рисунок 2.9 – Вікно після вибору 4 пункту

Якщо об'єкт із таким номером знайдено (Рис 2.10), поточний зайнятий об'єм складу `currentVolumeM3` зменшується на величину об'єму цієї деталі, об'єкт видаляється з пам'яті через деструктор, а сам вказівник усувається з вектора через `storage.erase()`.

```
Ваш вибір: 4
Введіть артикул запчастини для видачі: BRK-099
Запчастина №BRK-099 (Гальмівні колодки) успішно видана зі складу.

>>> ГОЛОВНЕ МЕНЮ <<<
1. Переглянути каталог заводу
2. Зробити запчастину та відправити на склад
3. Подивитись інформацію про склад
4. Забрати запчастину зі складу
5. Вихід
Ваш вибір: █
```

Рисунок 2.10 – Вікно у випадку успішної видачі зі складу

Якщо деталі немає, додаток безпечно перехоплює помилку `invalid_argument` і повідомляє, що запчастина не знайдена (Рис 2.11).

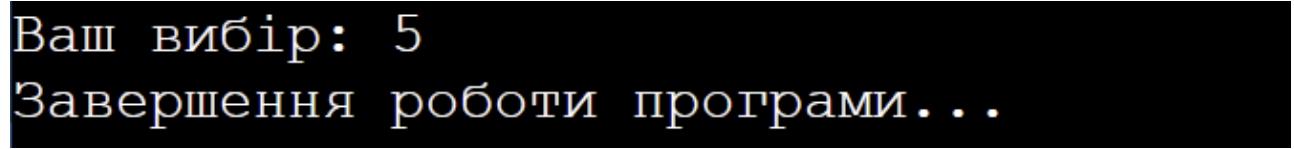
```
Ваш вибір: 4
Введіть артикул запчастини для видачі: BRK-099
[ЗНАЙДЕНА ПОМИЛКА] Запчастина з артикулом BRK-099 на складі не знайдена!

>>> ГОЛОВНЕ МЕНЮ <<<
1. Переглянути каталог заводу
2. Зробити запчастину та відправити на склад
3. Подивитись інформацію про склад
4. Забрати запчастину зі складу
5. Вихід
Ваш вибір: █
```

Рисунок 2.11 – Вікно у випадку, якщо запчастину на складі не знайдено

Вибір 5: Вихід:

При натисканні кнопки «5» додаток виводить фінальне повідомлення «Завершення роботи програми...» (Рис 2.12), і повертає 0 у функції main, що призводить до закриття консолі.



```
Ваш вибір: 5
Завершення роботи програми...
```

Рисунок 2.12 – Вікно після вибору 5 пункту

Перед закриттям викликається деструктор класу Warehouse, який автоматично очищує всю динамічну пам'ять, виділену під залишки запчастин на складі.

## ВИСНОВКИ

У ході виконання курсової роботи було розроблено програму на мові C++ з використанням принципів об'єктно-орієнтованого програмування на тему «Запчастини». У процесі роботи було створено ієрархію класів для представлення деталей, готових продуктів, каталогу заводу та складського обліку, а також реалізовано взаємодію між ними.

Під час розробки програми були використані основні принципи ООП: інкапсуляція, наслідування та поліморфізм. Для кожного класу були створені відповідні закриті та захищені поля, конструктори й методи для роботи з об'єктами. Також у програмі реалізовано автоматичне конвертування одиниць виміру (грамів у кілограми та кубічних сантиметрів у кубічні метри), механізм динамічного виділення пам'яті за допомогою вектора вказівників та надійну систему обробки виняткових ситуацій у разі переповнення місткості складу або введення неіснуючого артикула деталі.

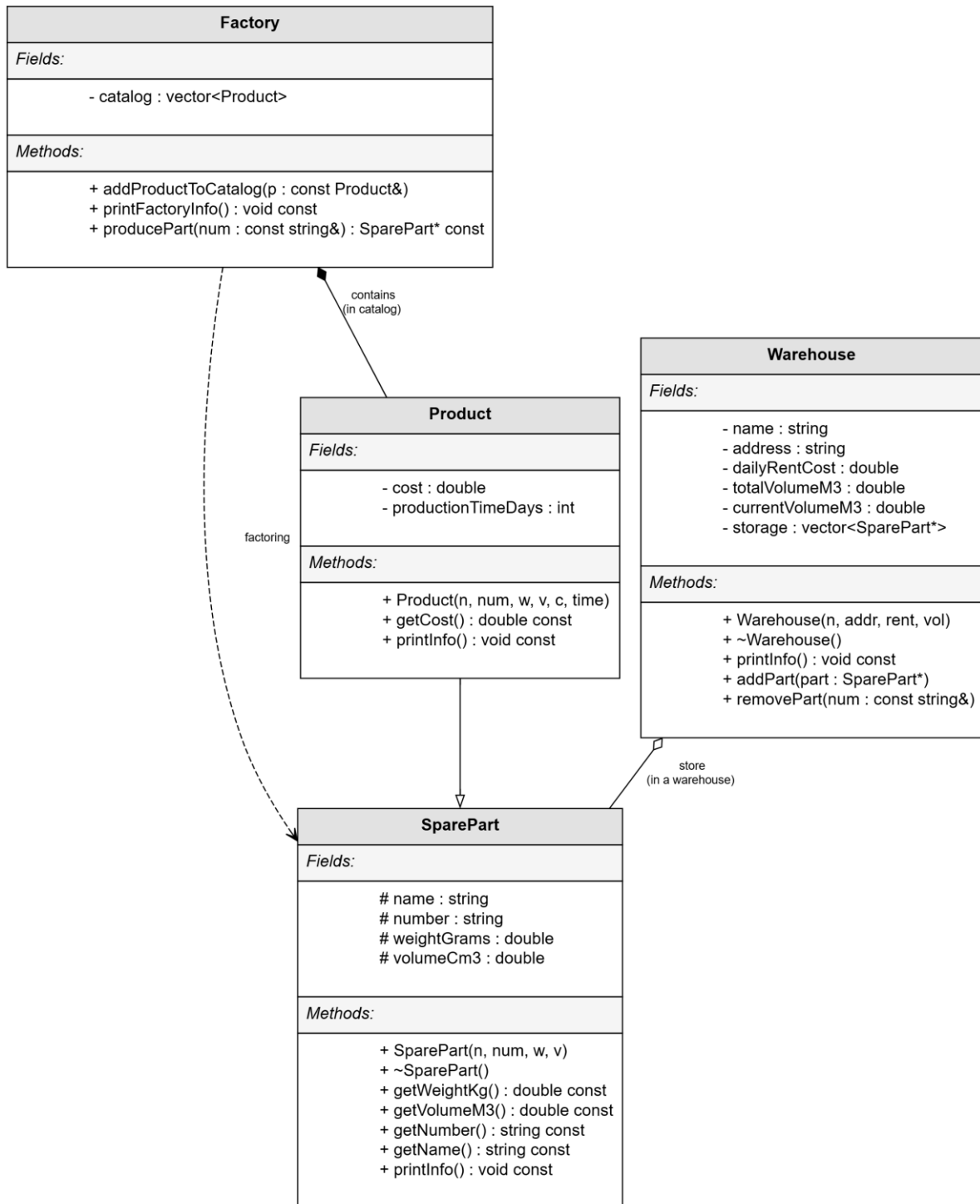
У результаті виконання курсової роботи були закріплені практичні навички програмування на C++, роботи з класами, конструкторами, методами та консольним інтерфейсом. Розроблена програма демонструє можливість створення повноцінної системи управління виробничими та складськими процесами засобами об'єктно-орієнтованого програмування.

## СПИСОК ЛІТЕРАТУРИ

1. Cppreference.com. Класи та об'єкти у C++ (Офіційна документація). [Електронний ресурс] – Режим доступу: <https://uk.cppreference.com/w/cpp/language/classes>
2. Web Library. Основні поняття ООП: інкапсуляція, поліморфізм, успадкування. [Електронне видання] / 2026. – Режим доступу: <http://weblib.com.ua/blog/osnovni-ponyattya-oop>
3. W3Schools. C++ OOP (Object-Oriented Programming). [Електронний ресурс] – Режим доступу: [https://www.w3schools.com/cpp/cpp\\_oop.asp](https://www.w3schools.com/cpp/cpp_oop.asp)
4. Репозиторій GitHub – Режим доступу: <https://github.com/nataliaherashchenko-gif/Coursework>



## Додаток А. Діаграма класів



## Додаток Б. Код основних класів

```
#include <iostream>
#include <string>
#include <vector>
#include <limits>
#include <iomanip>
#include <stdexcept>

using namespace std;

class SparePart {
protected:
    string name;
    string number;
    double weightGrams;
    double volumeCm3;

public:
    SparePart(string n, string num, double w, double v) {
        name = n;
        number = num;
        weightGrams = w;
        volumeCm3 = v;
    }

    virtual ~SparePart() {}

    double getWeightKg() const { return weightGrams / 1000.0; }
    double getVolumeM3() const { return volumeCm3 / 1000000.0; }
    string getNumber() const { return number; }
    string getName() const { return name; }

    virtual void printInfo() const {
        cout << left << setw(20) << name << " | № " << setw(10) << number
              << " | Бес: " << setw(6) << getWeightKg() << " кг | Об'єм: "
              << setw(6) << getVolumeM3() << " м^3";
    }
};
```

Лістинг Б.1 – клас SparePart

```
class Product : public SparePart {
private:
    double cost;
    int productionTimeDays;

public:
    Product(string n, string num, double w, double v, double c, int time)
        : SparePart(n, num, w, v) {
```

Лістинг Б.2 – Клас Product, аркуш 1

```

        cost = c;
        productionTimeDays = time;
    }

    double getCost() const { return cost; }

    void printInfo() const {
        SparePart::printInfo();
        cout << " | Вартість: $" << setw(6) << cost
            << " | Виробництво: " << productionTimeDays << " дн.";
    }
};

class Product : public SparePart {
private:
    double cost;
    int productionTimeDays;

public:
    Product(string n, string num, double w, double v, double c, int time)
        : SparePart(n, num, w, v) {
        cost = c;
        productionTimeDays = time;
    }

    double getCost() const { return cost; }

    void printInfo() const {
        SparePart::printInfo();
        cout << " | Вартість: $" << setw(6) << cost
            << " | Виробництво: " << productionTimeDays << " дн.";
    }
};

```

### Лістинг Б.2 – Клас Product, аркуш 2

```

class Factory {
private:
    vector<Product> catalog;

public:
    void addProductToCatalog(const Product& p) {
        catalog.push_back(p);
    }

    void printFactoryInfo() const {
        cout << "\n--- КАТАЛОГ ЗАВОДУ ---\n";
        if (catalog.empty()) {
            cout << "Каталог порожній.\n";
            return;
        }
        for (size_t i = 0; i < catalog.size(); ++i) {
            catalog[i].printInfo();
            cout << "\n";
        }
    }
};

```

### Лістинг Б.3 – Клас Factory, аркуш 1

```

        cout << "-----\n";
    }

    SparePart* producePart(const string& num) const {
        for (size_t i = 0; i < catalog.size(); ++i) {
            if (catalog[i].getNumber() == num) {
                return new Product(catalog[i]);
            }
        }
        throw invalid_argument("Запчастина з артикулом " + num + " не знайдена в каталозі заводу!");
    }
};

```

### Лістинг Б.3 – Клас Factory, аркуш 2

```

class Warehouse {
private:
    string name;
    string address;
    double dailyRentCost;
    double totalVolumeM3;
    double currentVolumeM3;
    vector<SparePart*> storage;

public:
    Warehouse(string n, string addr, double rent, double vol) {
        name = n;
        address = addr;
        dailyRentCost = rent;
        totalVolumeM3 = vol;
        currentVolumeM3 = 0.0;
    }

    ~Warehouse() {
        for (size_t i = 0; i < storage.size(); ++i) {
            delete storage[i];
        }
    }

    void printInfo() const {
        cout << "\n=== ІНФОРМАЦІЯ ПРО СКЛАД ===\n";
        cout << "Назва: " << name << " | Адреса: " << address << "\n";
        cout << "Оренда: $" << dailyRentCost << "/доба\n";
        cout << "Об'єм: " << currentVolumeM3 << " / " << totalVolumeM3 << " м^3\n";
        cout << "Запчастин у наявності: " << storage.size() << "\n";
        if (!storage.empty()) {
            cout << "\nВміст складу:\n";
            for (size_t i = 0; i < storage.size(); ++i) {
                storage[i]->printInfo();
                cout << "\n";
            }
        }
        cout << "=====\n";
    }
};

```

### Лістинг Б.4 – Клас Warehouse, аркуш 1

```

void addPart(SparePart* part) {
    if (part == nullptr) return;

    double partVol = part->getVolumeM3();
    if (currentVolumeM3 + partVol > totalVolumeM3) {
        delete part;
        throw overflow_error("Недостатньо місця на складі! Операцію скасовано.");
    }

    currentVolumeM3 += partVol;
    storage.push_back(part);
    cout << "Запчастина успішно додана на склад.\n";
}

void removePart(const string& num) {
    for (auto it = storage.begin(); it != storage.end(); ++it) {
        if ((*it)->getNumber() == num) {
            currentVolumeM3 -= (*it)->getVolumeM3();
            cout << "Запчастина №" << num << " (" << (*it)->getName() << " " << "успішно видана зі складу.\n";

            delete *it;
            storage.erase(it);
            return;
        }
    }
    throw invalid_argument("Запчастина з артикулом " + num + " на складі не знайдена!");
}
};

```

#### Лістинг Б.4 – Клас Warehouse, аркуш 2

```

void clearInput() {
    cin.clear();
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
}

int main() {
    setlocale(LC_ALL, "UKRAINIAN");

    Factory factory;
    factory.addProductToCatalog(Product("Двигун V8", "ENG-001", 150000, 2000000, 5000, 14));
    factory.addProductToCatalog(Product("Гальмівні колодки", "BRK-099", 1500, 5000, 45, 1));
    factory.addProductToCatalog(Product("Фара передня", "LGH-102", 2500, 15000, 120, 2));

    Warehouse warehouse("Центральний Хаб", "Промислова 12", 200.0, 5.0);

    int choice = 0;
    while (true) {

```

#### Лістинг Б.5 – Головна функція програми (main), аркуш 1

```

cout << "\n>>> ГОЛОВНЕ МЕНЮ <<<\n";
cout << "1. Переглянути каталог заводу\n";
cout << "2. Зробити запчастину та відправити на склад\n";
cout << "3. Подивитись інформацію про склад\n";
cout << "4. Забрати запчастину зі складу\n";
cout << "5. Вихід\n";
cout << "Ваш вибір: ";
if (!(cin >> choice)) {
    cout << "[ПОМИЛКА] Введіть число!\n";
    clearInput();
    continue;
}

try {
    switch (choice) {
        case 1:
            factory.printFactoryInfo();
            break;
        case 2: {
            string art;
            cout << "Введіть артикул (номер) запчастини з каталога: ";
            cin >> art;

            SparePart* newPart = factory.producePart(art);
            warehouse.addPart(newPart);
            break;
        }
        case 3:
            warehouse.printInfo();
            break;
        case 4: {
            string art;
            cout << "Введіть артикул запчастини для видачі: ";
            cin >> art;
            warehouse.removePart(art);
            break;
        }
        case 5:
            cout << "Завершення роботи програми...\n";
            return 0;
        default:
            cout << "[ПОМИЛКА] Невірний пункт меню.\n";
    }
}
catch (const exception& e){
    cout << "[ЗНАЙДЕНА ПОМИЛКА] " << e.what() << "\n";
}

return 0;
}

```

Лістинг Б.5 – Головна функція програми (main), аркуш 2